PROJECT REPORT

1. PROJECT TITLE

DarshanEase — Temple Darshan Slot Booking & Donation Management System**

---

2. PROJECT DESCRIPTION

System Overview

DarshanEase is a full-stack web application that enables devotees to book darshan slots at temples and make donations. It provides temple administrators with tools to manage temples, slots, bookings, cancellations, and donation records.

Problem Domain

- Manual temple visit management leads to overcrowding and poor visitor experience
- No centralized platform for slot booking and donation tracking
- Difficulty in managing temple schedules and capacity
- Lack of transparency in donation handling and temple administration

Core Technology Used

- Frontend: React 19, React Router, Axios, Bootstrap 5
- Backend: Node.js, Express 5
- Database: MongoDB with Mongoose ODM
- Auth: JWT (JSON Web Tokens), bcryptjs

Key Capabilities

1. User registration and authentication (JWT-based)
2. Temple listing with images and descriptions
3. Darshan slot creation and management (admin)
4. Booking slots by date, time, and number of persons
5. Soft cancellation of bookings
6. Donation flow per temple with optional message
7. My Bookings and My Donations for users
8. Admin: temples CRUD, recent bookings, recent cancellations, donation statistics

Target Users

- ✓ Devotees: End-users booking darshan slots and making donations
- ✓ Temple Administrators: Manage temples, slots, and view analytics

Real-world Relevance

Applicable to religious institutions, pilgrimage sites, and heritage temples seeking digitized booking and donation management.

---

3. APPLICATION SCENARIOS / USE CASES

- ✓ Devotee Usage: A user registers, browses temples, views available slots by temple and date, books a slot for a family of 4, and later cancels if plans change. The user also donates to a temple with an optional message.
- ✓ Temple Administrator Usage: An admin logs in, adds a new temple with name, location, description, and image URL. The admin creates darshan slots (date, time, available seats) and monitors recent bookings, cancellations, and donation statistics.
- ✓ Educational Usage: Students can study the MERN stack implementation, JWT authentication, role-based access control, and RESTful API design.
- ✓ Enterprise / Institutional Usage: Temple trusts or religious bodies can deploy the system for multiple temples under their management.

---

4. TECHNICAL ARCHITECTURE OVERVIEW

High-level Architecture Diagram

Client-Server Architecture

Component	Description
UI (React Frontend)	User interface built with React
API (Express API)	RESTful API built with Express
Auth (Auth Middleware)	Middleware for authentication and authorization
Mongo (MongoDB)	NoSQL database for data storage
UI --> API	HTTP/REST requests from UI to API
API --> Auth	API forwards requests to Auth middleware
Auth --> API	Auth middleware returns valid token to API
API <--> Mongo	API interacts with MongoDB for data

Made with Napkin

Component Interaction Diagram

Frontend vs. Backend

Characteristic	Frontend	Backend
Components	Pages, AuthContext, Axios API	Express Routes, Middleware, Controllers, Models
Interaction	Pages use AuthContext and API	API uses Bearer Token for Routes
Flow	Routes use Middleware, Middleware uses Controllers, Controllers use Models	

Made with Napkin

Data / Request Flow

- ✓ Frontend layer: React SPA with React Router; AuthContext holds user and token; Axios sends requests with `Authorization: Bearer <token>`.
- ✓ Backend layer: Express routes → auth middleware → role middleware → controllers → Mongoose models.
- ✓ Database layer: MongoDB stores Users, Temples, DarshanSlots, Bookings, Donations.
- ✓ External APIs / Services: None; uses direct MongoDB connection.
- ✓ Deployment environment: Local development (localhost:3000 frontend, localhost:5000 backend); can be deployed to cloud (e.g., Render, MongoDB Atlas).

---

5. PREREQUISITES

5.1 Software Requirements

- ✓ Development tools: Node.js (v18+ recommended), npm or yarn
- ✓ IDE: VS Code, Cursor, or any modern code editor
- ✓ Runtime environment: Node.js runtime for backend; browser for frontend

- ✓ Cloud services (if any): MongoDB Atlas (optional, for cloud DB) or local MongoDB

5.2 Libraries / Frameworks / Dependencies

Backend (`package.json`):

Node.js Package Overview

Package	Version	Purpose
express	^5.2.1	Web framework
mongoose	^9.2.4	MongoDB ODM
bcryptjs	^3.0.3	Password hashing
jsonwebtoken	^9.0.3	JWT auth
cors	^2.8.6	CORS handling
dotenv	^17.3.1	Env config
morgan	^1.10.1	HTTP logging
nodemon	^3.1.14	Dev auto-reload

Made with Napkin

Frontend (`client/package.json`):

Package Information

Package	Version	Purpose
react	^19.2.4	UI library
react-dom	^19.2.4	React DOM
react-router-dom	^7.13.1	Routing
axios	^1.13.6	HTTP client
bootstrap		[Styling framework for web pages]

Made with Napkin

5.3 Hardware Requirements

- ✓ Minimum: 4GB RAM, dual-core processor
- ✓ Recommended: 8GB RAM for comfortable development
- ✓ No special devices or sensors required

---

6. PRIOR KNOWLEDGE REQUIRED

- ✓ Programming language: JavaScript (ES6+)
- ✓ Framework basics: React (components, hooks, context), Express (routes, middleware)
- ✓ Database fundamentals: NoSQL, MongoDB basics, Mongoose schemas
- ✓ Networking: REST APIs, HTTP methods, JWT authentication
- ✓ Cloud concepts: Optional for deployment (MongoDB Atlas, hosting platforms)

---

7. PROJECT OBJECTIVES

- ✓ Technical objectives: Build a secure, role-based MERN stack application with JWT auth, CRUD operations, and soft-delete patterns.
- ✓ Performance objectives: Fast page loads, responsive UI, minimal API round-trips.
- ✓ Deployment objectives: Run locally and be deployable to cloud platforms.
- ✓ Learning outcomes: Full-stack development, RESTful API design, authentication, authorization, and database modeling.

---

8. SYSTEM WORKFLOW

- ✓ User interaction: User accesses the web app, registers or logs in.
- ✓ Input handling: Login/register forms submit credentials; slot selection sends date, time, persons; donation form sends amount and optional message.
- ✓ Processing logic: Backend validates token, checks roles, validates inputs, updates MongoDB.
- ✓ Core system execution: Controllers create/update/delete records; mongoose models persist data.
- ✓ Output generation: API returns JSON responses (success/error messages, lists, objects).
- ✓ Response delivery: Frontend updates UI, shows success/error via modals or navigation.

---

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

Temples need a digital system for darshan slot booking and donation management to improve visitor experience and administrative control.

9.2 Functional Requirements

1. User registration and login
2. Temple listing and management (admin)
3. Slot creation and deletion (admin)
4. Slot listing by temple and by temple+date
5. Booking creation with numberOfPersons
   - ✓ Booking cancellation (soft delete)
   - ✓ Donation creation with amount and optional message
   - ✓ View my bookings and my donations
   - ✓ Admin: recent bookings, recent cancellations, donation stats

9.3 Non-Functional Requirements

- ✓ Secure authentication (JWT, hashed passwords)
- ✓ Role-based access (USER, ADMIN)

- ✓ Responsive UI
- ✓ CORS enabled for frontend-backend communication

9.4 System Design Decisions

1. JWT for stateless auth; token stored in localStorage
2. Soft delete for bookings (status: BOOKED | CANCELLED, cancelledAt)
3. Admin identified by email (e.g., admin@gmail.com) or role field
4. Mongoose schemas with references for User, Temple, DarshanSlot, Booking, Donation

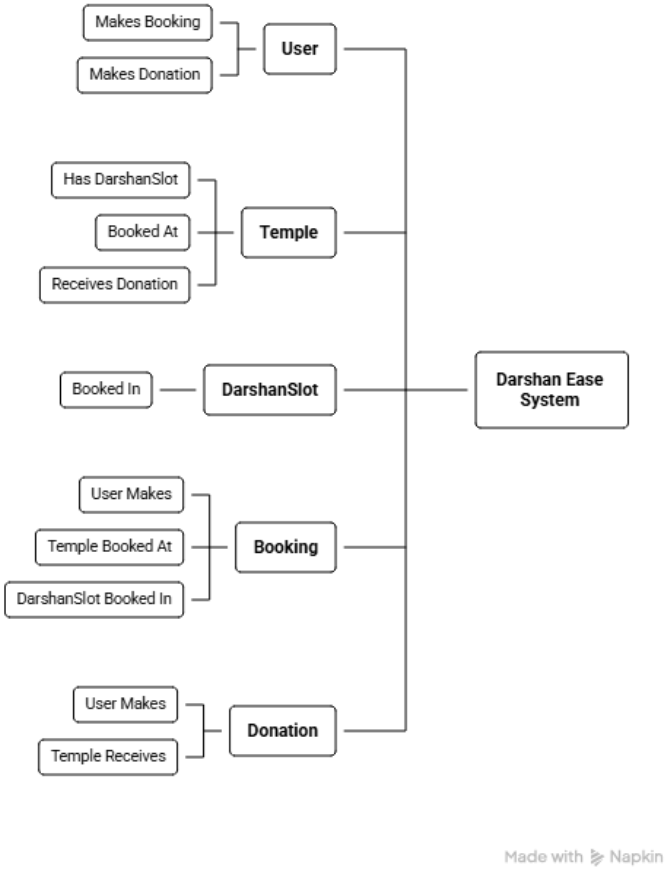
9.5 Technology Stack Selection Justification

1. React: Component-based UI, large ecosystem, easy state management
2. Express: Lightweight, flexible, widely used for Node.js APIs
3. MongoDB + Mongoose: Flexible schema, easy prototyping, good for JSON-like data
4. JWT: Stateless, scalable auth across services
5. Bootstrap: Rapid UI development, responsive layout

Architecture Diagram

(See Section 4)

Darshan Ease System Architecture



User {

ObjectId _id PK

string name

string email

string password

string role

}

Temple {

ObjectId _id PK

string name

```
    string location
    string description
    string imageUrl
}

DarshanSlot {
    ObjectId _id PK
    ObjectId temple FK
    Date date
    string time
    number availableSeats
}

Booking {
    ObjectId _id PK
    ObjectId user FK
    ObjectId temple FK
    ObjectId slot FK
    number numberOfPersons
    string status
    Date cancelledAt
}

Donation {
    ObjectId _id PK
    ObjectId temple FK
    ObjectId user FK
    number amount
    string message
```

}

API Structure

API Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Register user
POST	/api/auth/login	No	Login
GET	/api/temples	Yes	List temples
GET	/api/temples/:id	Yes	Get temple by ID
POST	/api/temples	Admin	Create temple
PUT	/api/temples/:id	Admin	Update temple
DELETE	/api/temples/:id	Admin	Delete temple
POST	/api/slots	Admin	Create slot
GET	/api/slots/:templeId	Yes	Slots by temple
GET	/api/slots/:templeId/:date	Yes	Slots by temple and date
DELETE	/api/slots/:id	Admin	Delete slot
POST	/api/bookings	Yes	Create booking
GET	/api/bookings/my	Yes	My bookings
DELETE	/api/bookings/:bookingId	Yes	Cancel booking
GET	/api/bookings/admin/recent	Admin	Recent bookings
GET	/api/bookings/admin/recent-cancellations	Admin	Recent cancellations
POST	/api/donations	Yes	Create donation
GET	/api/donations/me	Yes	My donations
GET	/api/donations/stats/summary	Admin	Donation stats

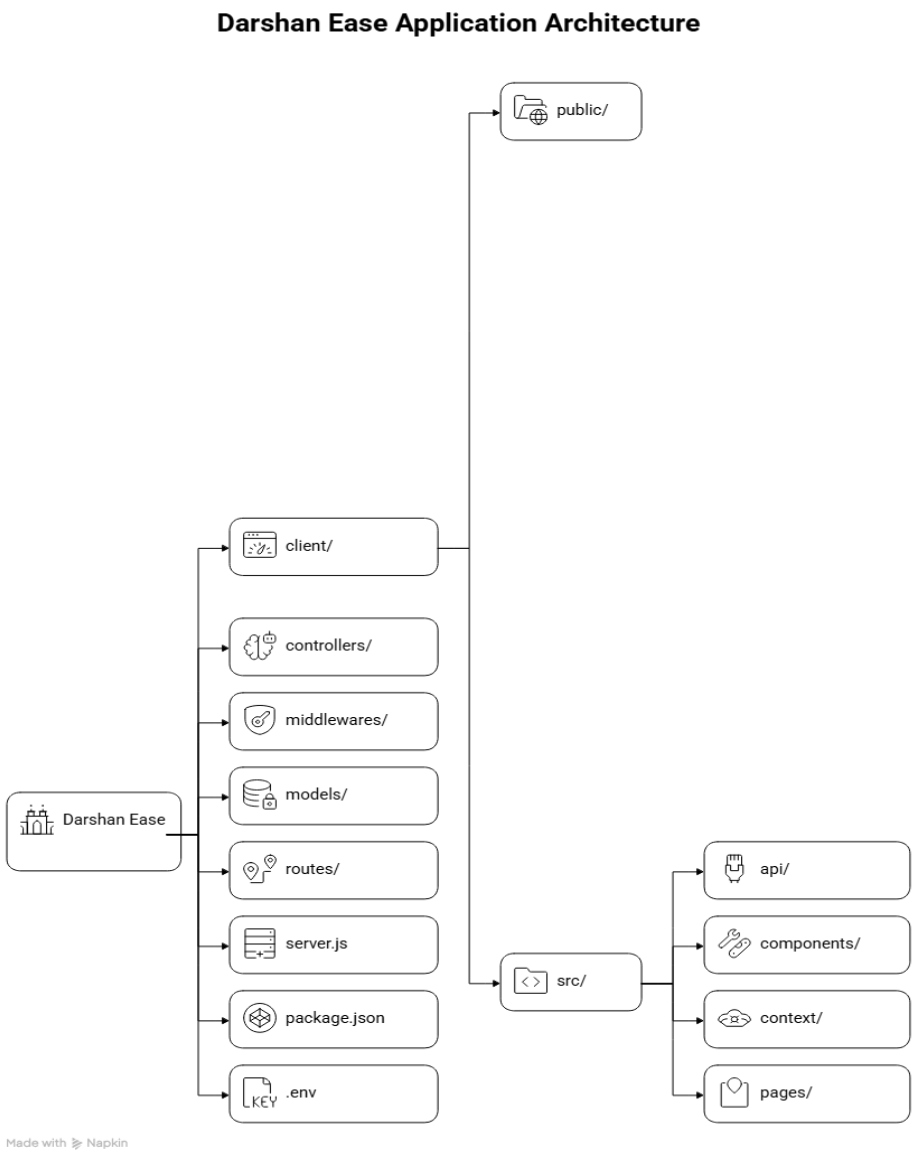
10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

1. Install Node.js (v18+)
2. Install MongoDB locally or create MongoDB Atlas cluster
3. Clone/create project folder

10.2 Dependency Installation

```bash

*Backend*

npm install

***# Frontend***

cd client && npm install

### 10.3 Project Structure Creation
```



10.4 Configuration Setup

- ✓ `.env` (backend): `MONGO_URI`, `JWT_SECRET`, `PORT`
- ✓ `client/src/api/axios.js`: baseURL `http://localhost:5000/api`, Bearer token in headers

---

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 – Authentication

- ✓ Description: JWT-based register/login; password hashed with bcryptjs.
- ✓ Code explanation: `authController` registers user (hashes password), logs in (checks credentials, returns JWT). Frontend stores `{ user, token }` in localStorage via AuthContext.

- ✓ Implementation: `controllers/authController.js`, `routes/authRoutes.js`, `middlewares/authMiddleware.js`, `context/AuthContext.js`

11.2 Feature 2 – Temple & Slot Management

- ✓ Description: CRUD temples and slots; slots linked to temple and date.
- ✓ Code explanation: `templeController` and `slotController` handle create/read/update/delete. Only admin can create/update/delete temples and slots.
- ✓ Implementation: `models/Temple.js`, `models/DarshanSlot.js`, `controllers/templeController.js`, `controllers/slotController.js`, `routes/templeRoutes.js`, `routes/slotRoutes.js`

11.3 Feature 3 – Booking & Donation

- ✓ Description: Users book slots (numberOfPersons); can cancel (soft delete). Users donate to temples with amount and optional message.
- ✓ Code explanation: `bookingController` creates booking, fetches my bookings, cancels (sets status CANCELLED, cancelledAt). `donationController` creates donation, fetches my donations, admin stats (total amount, count, temple breakdown).
- ✓ Implementation: `models/Booking.js`, `models/Donation.js`, `controllers/bookingController.js`, `controllers/donationController.js`, `routes/bookingRoutes.js`, `routes/donationRoutes.js`

Database Schema

(See Section 9.4 ER Diagram)

API Endpoints

(See Section 9.5 API Structure)

---

12. MILESTONE 4: INTEGRATION & OPTIMIZATION

12.1 Component Integration

1. Navbar and Footer wrap all routes; AuthContext provides user/token to pages.
2. Axios instance configured with baseURL and optional Bearer token injection.
3. React Router routes map to pages (Home, Login, Register, Slots, BookSlot, Donate, MyBookings, MyDonations, Admin, AdminRecentBookings, AdminRecentCancellations).

12.2 Performance Optimization

1. Minimal re-renders; useEffect dependencies set correctly.
2. Placeholder images and error fallbacks for temple images.
3. Bootstrap for responsive layout and quick UI development.

12.3 Security Enhancements

1. Passwords hashed with bcryptjs.
2. JWT verification on protected routes.
3. Role-based middleware (`isAdmin`) for admin-only endpoints.
4. CORS enabled for allowed origins.

12.4 Error Handling & Validation

1. Try-catch in controllers; appropriate HTTP status codes (401, 403, 404, 500).
2. Mongoose validation on models (required fields, min values).
3. Frontend modal/toast for success and error messages.

---

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

Test Case Results

Test Case ID	Input	Expected Output	Status
TC01	Valid email, password (register)	User created, success	Pass
TC02	Invalid email (register)	Validation error	Pass
TC03	Valid email, password (login)	JWT returned	Pass
TC04	Wrong password (login)	401 Unauthorized	Pass
TC05	Create temple (admin)	Temple created	Pass
TC06	Create temple (user)	403 Forbidden	Pass
TC07	Book slot with valid slotId, persons	Booking created	Pass
TC08	Book slot with no token	401 Unauthorized	Pass
TC09	Cancel booking	status CANCELLED	Pass
TC10	Donate with amount	Donation created	Pass

Made with Napkin

13.2 Unit Testing

- ✓ React testing library available (`client/src/App.test.js`); can be extended for components.

13.3 Integration Testing

- ✓ API endpoints can be tested with Postman or similar tools.

- ✓ Frontend-backend integration verified manually.

13.4 User Acceptance Testing

1. User flows: register → login → browse temples → book slot → view my bookings → cancel if needed.
2. Donation flow: select temple → donate → view my donations.
3. Admin flows: manage temples, slots, view recent bookings/cancellations, donation stats.

13.5 Performance Metrics (Web/App)

1. Response time: < 500ms for typical API calls.
2. Load handling: Suitable for small-to-medium traffic.
3. Security validation: JWT and role checks enforced.

---

14. DEPLOYMENT

14.1 Deployment Architecture

- ✓ Frontend: Static build served from client/build or hosting CDN.
- ✓ Backend: Node.js server (Express) on cloud or VPS.
- ✓ Database: MongoDB (local or MongoDB Atlas).

14.2 Hosting Platform

- ✓ Frontend: Vercel, Netlify, or static hosting.
- ✓ Backend: Render, Railway, Heroku, or VPS.
- ✓ Database: MongoDB Atlas (cloud).

14.3 Deployment Steps

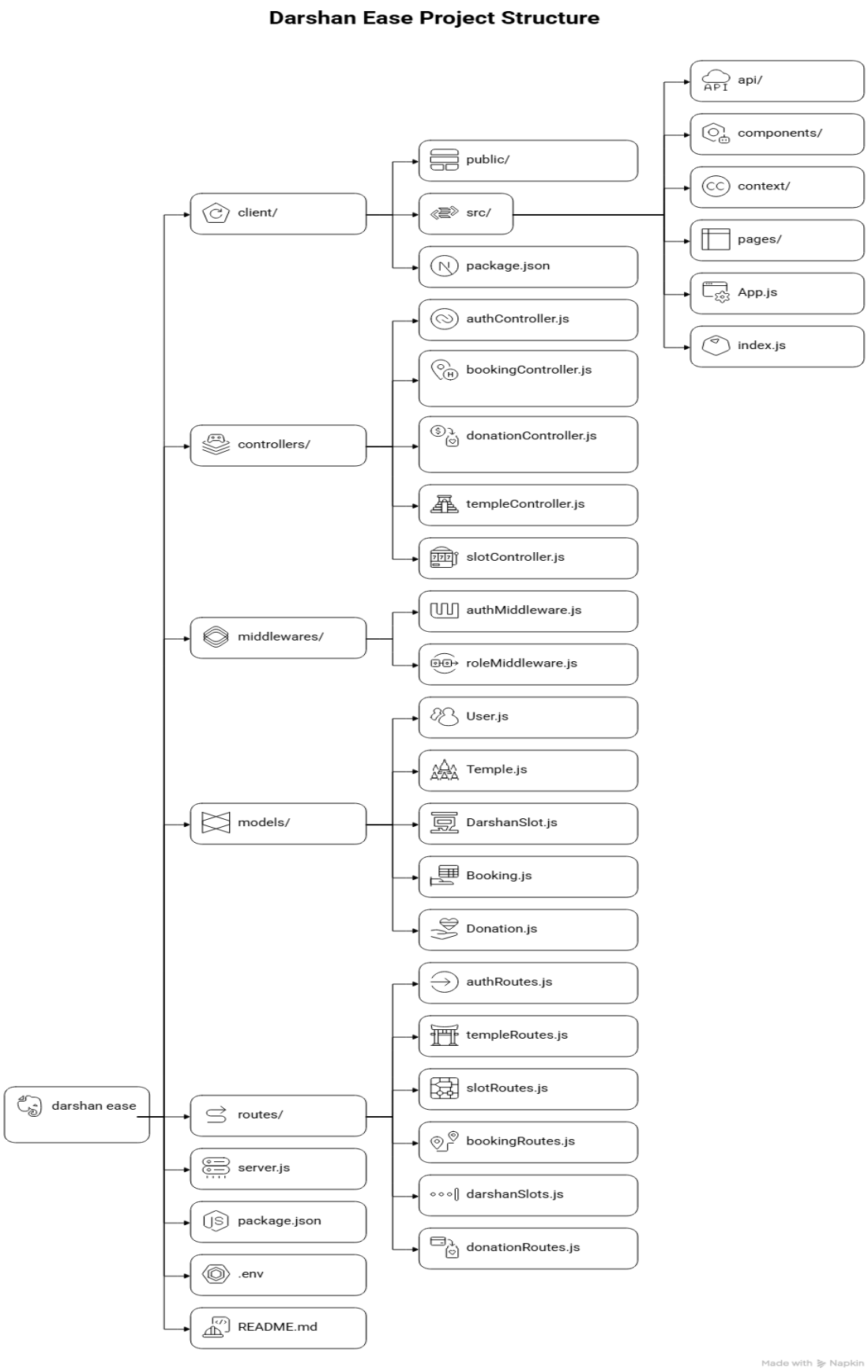
1. Set environment variables on hosting platform (MONGO_URI, JWT_SECRET, PORT).
2. Build frontend: `cd client && npm run build`.
3. Deploy backend with `npm start`.
4. Point frontend baseURL to deployed API URL.

14.4 Production Considerations

1. Use strong JWT_SECRET.
2. Enable HTTPS.
3. Set CORS origins to deployed frontend domain.
4. Use MongoDB Atlas with IP whitelist or VPC peering.

---

15. PROJECT STRUCTURE



FOLDER STRUCTURE

- `client/`: React SPA; `api/` for HTTP client; `components/` for reusable UI; `context/` for auth state; `pages/` for route components.
- `controllers/`: Business logic for each domain.
- `middlewares/`: Auth and role checks.
- `models/`: Mongoose schemas.
- `routes/`: Express route definitions.

---

16. RESULTS

System Output

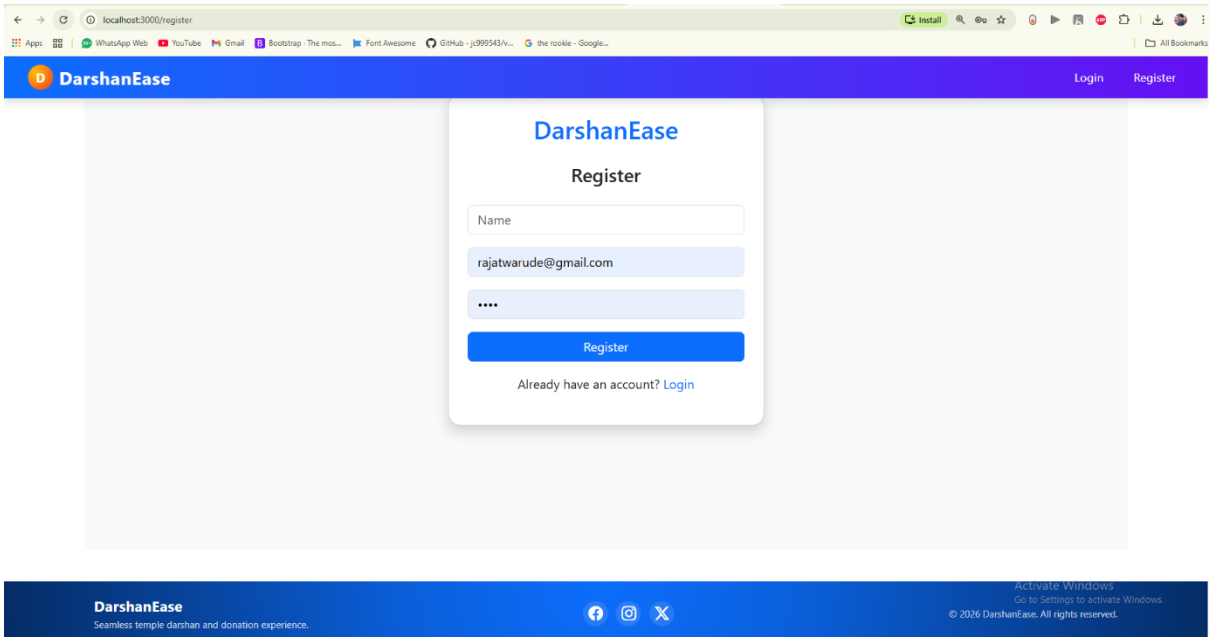
1. Functional web application with temple listing, slot booking, and donations.
2. Admin panel for temple/slot management and analytics.
3. Responsive UI with Bootstrap.

Performance Evaluation

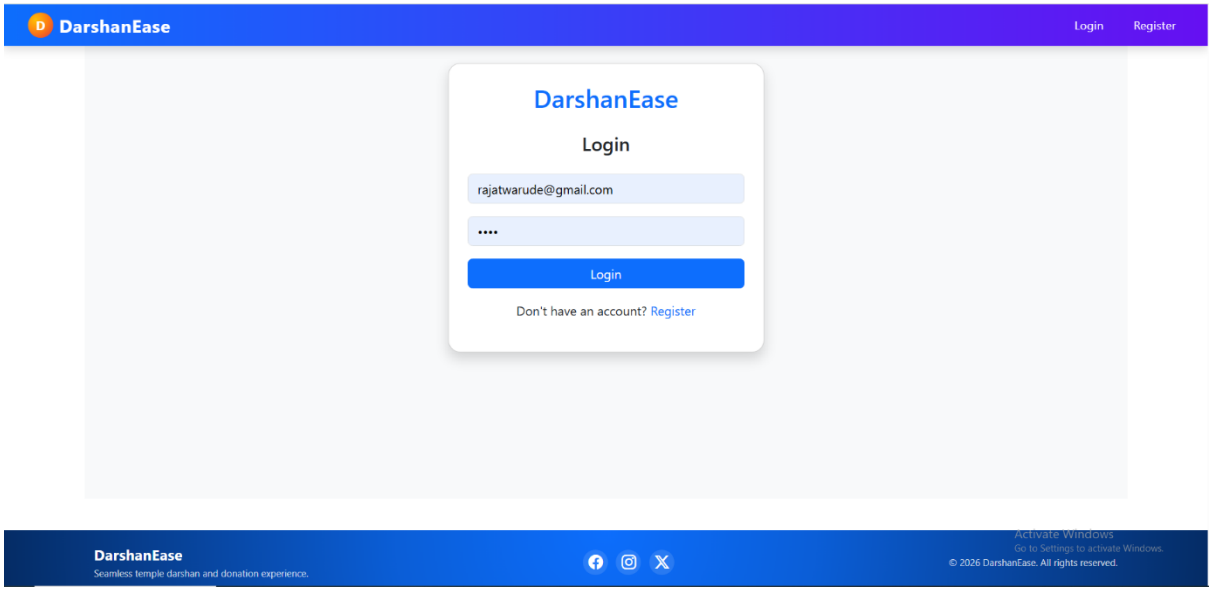
1. APIs respond quickly under normal load.
2. Frontend loads in under a few seconds.

Screenshots

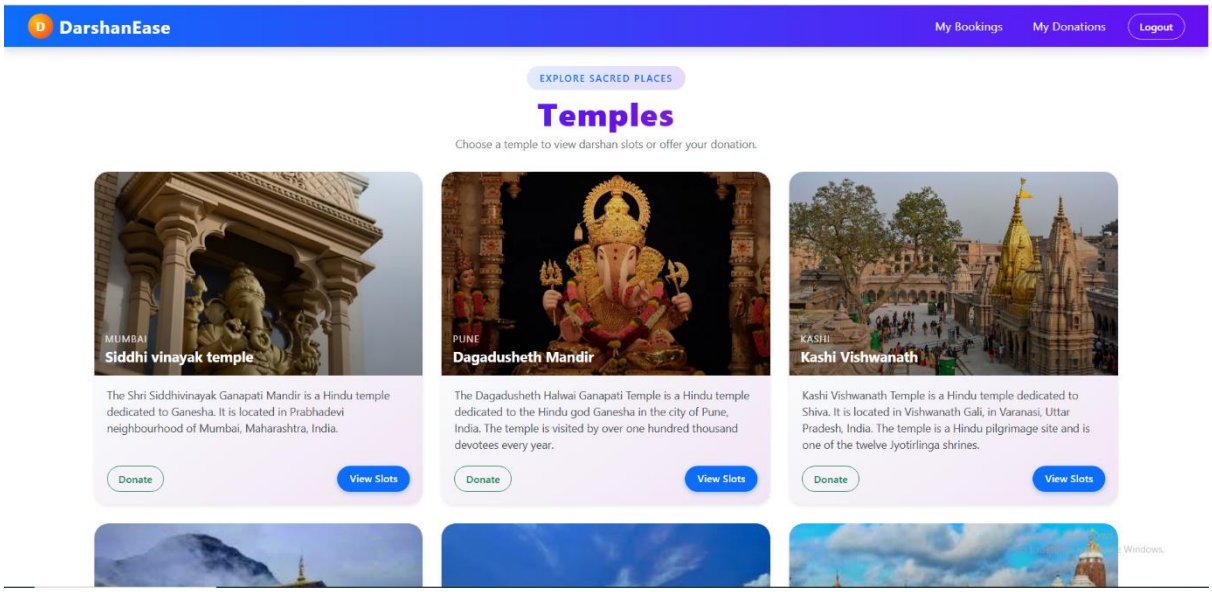
Register



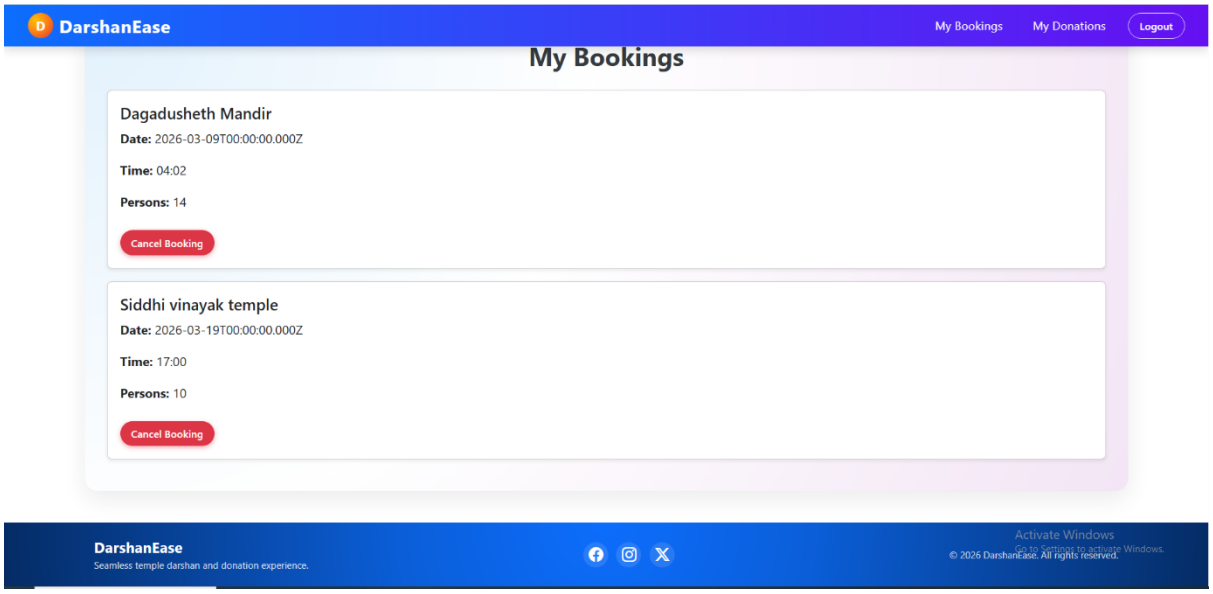
Login



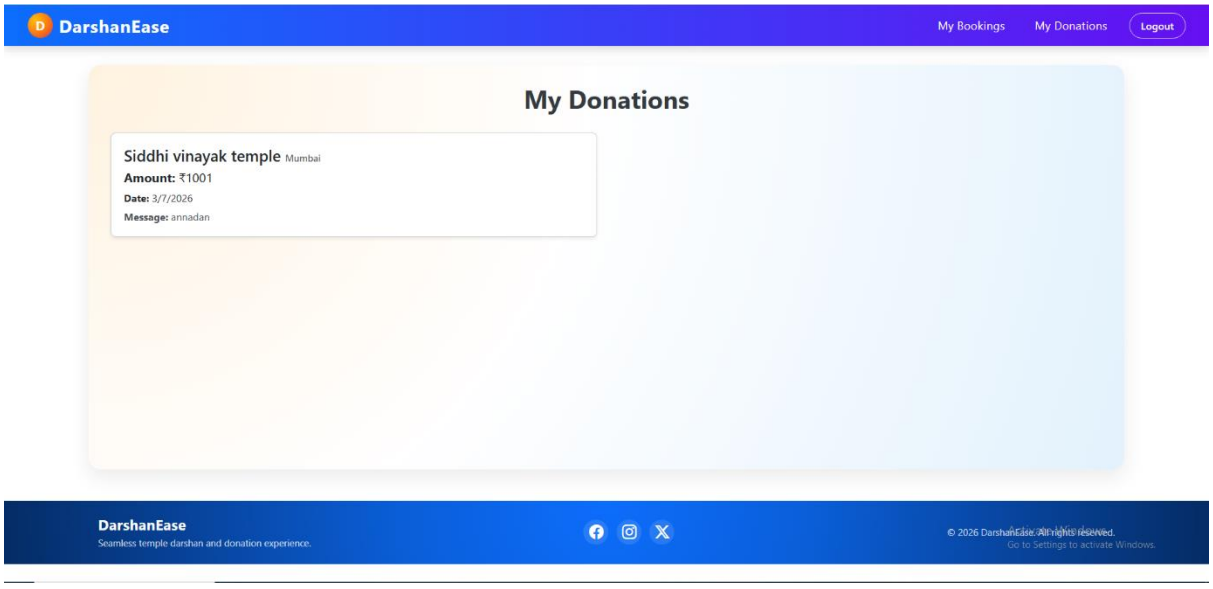
Homepage



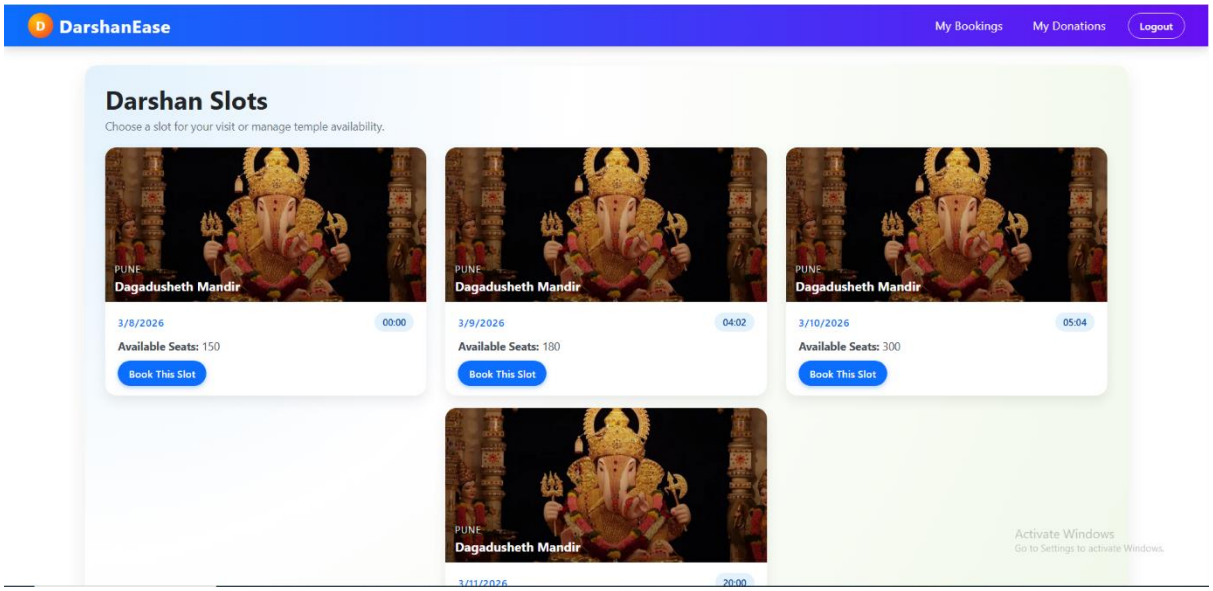
MyBookings



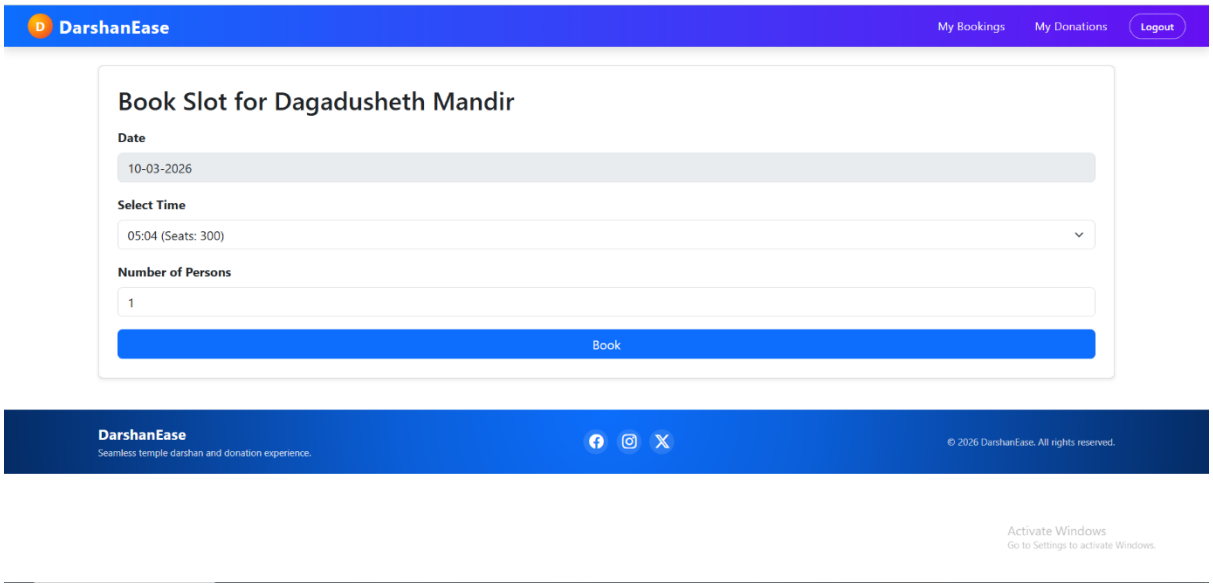
Mydonation



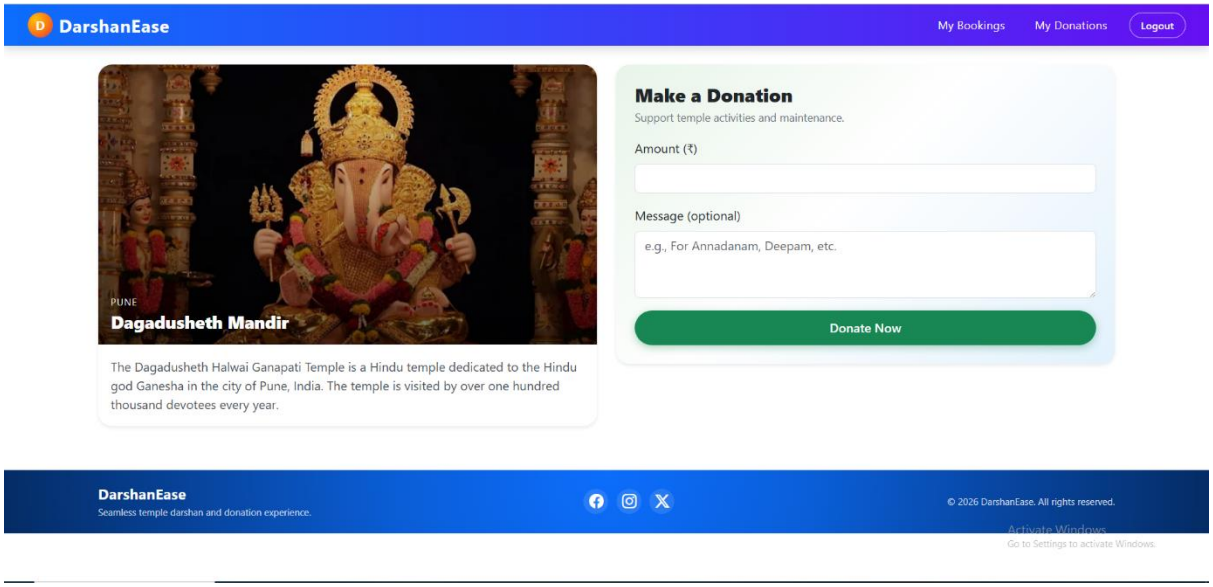
Bookslot



Bookslotform

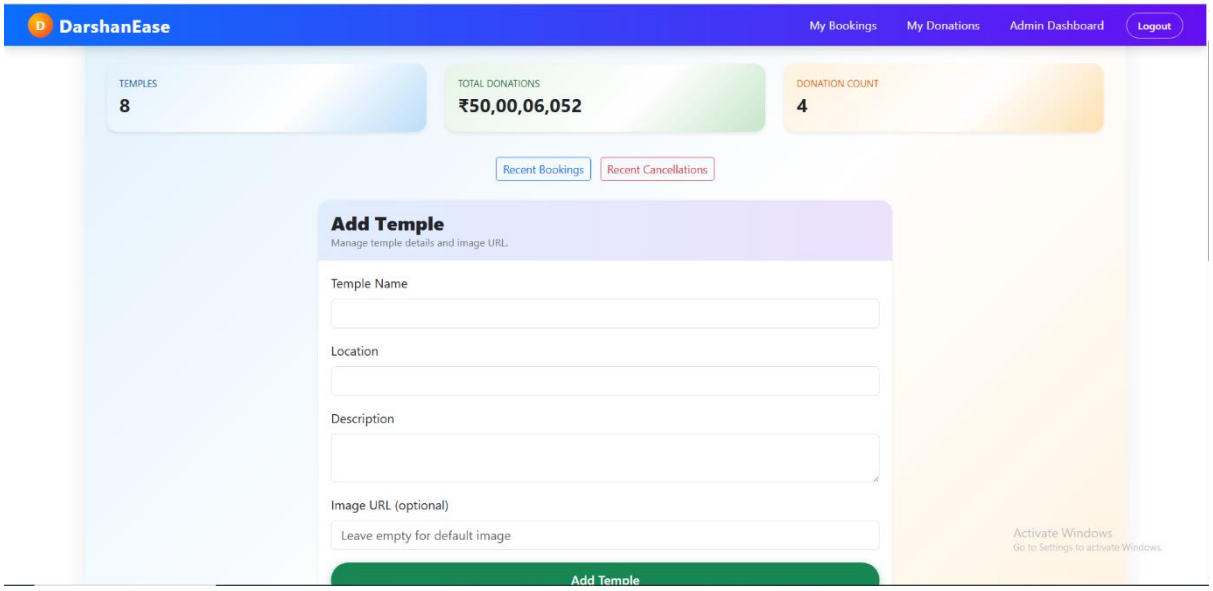


Donationpage

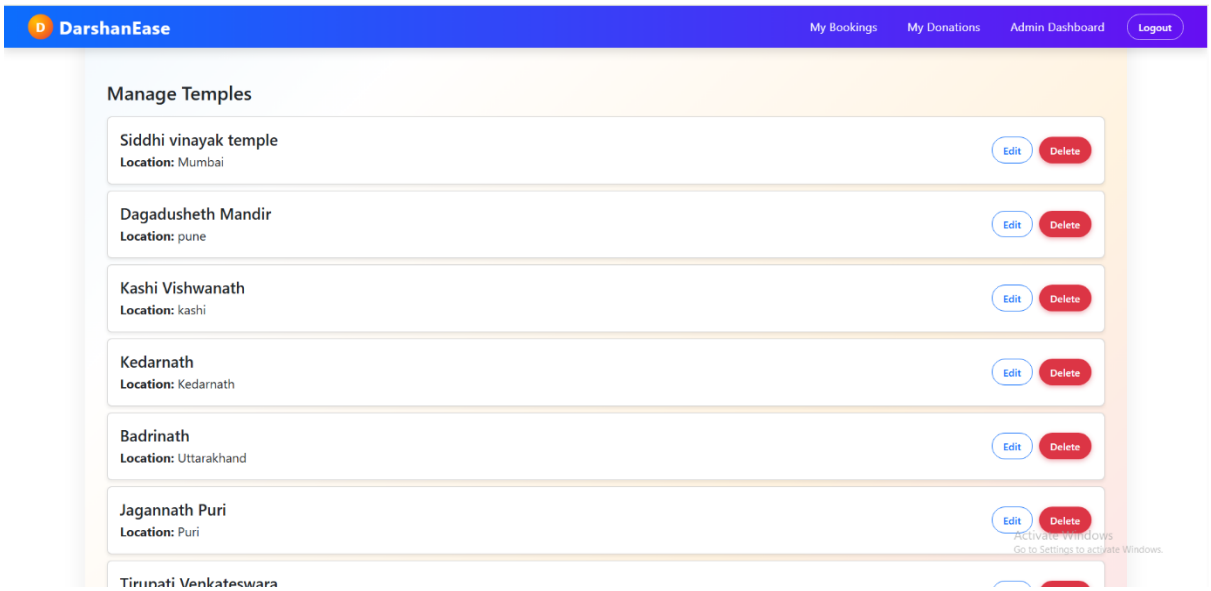


Admin button

Admindashboard



Edit temple



Benchmark Results

(Add response time or load test results if available.)

---

17. ADVANTAGES & LIMITATIONS

Advantages

1. Centralized booking and donation management.
2. Role-based access for users and admins.
3. Soft cancellation preserves history.
4. Clean separation of frontend and backend.
5. Scalable to multiple temples.

Limitations

1. No payment gateway integration (donations are recorded only).
2. No email notifications for booking/cancellation.
3. No slot capacity auto-update on concurrent bookings (requires additional concurrency handling).
4. Single admin model; no multi-temple admin roles.

---

18. FUTURE ENHANCEMENTS

1. Scalability:** Redis for session/cache; queue for email notifications.
2. Feature expansion:** Payment integration (Razorpay/Stripe), SMS/email alerts, calendar view for slots.
3. Cloud integration:** Deploy on AWS/GCP; use serverless functions for heavy tasks.
4. Mobile integration:** React Native or PWA for mobile users.
5. Automation:** Cron jobs for slot generation; automated reminders.

---

19. CONCLUSION

DarshanEase successfully implements a temple darshan slot booking and donation management system using the MERN stack. It demonstrates JWT authentication, role-based authorization, RESTful API design, and a responsive React frontend. The system is suitable for real-world deployment with minor enhancements and can serve as a foundation for religious institutions seeking digital transformation.

---

20. APPENDIX

Source Code

- Located in project folder `darshan ease/`.

Configuration Files

- `.env`: MONGO_URI, JWT_SECRET, PORT

- `client/package.json`, `package.json`: Dependencies

API Documentation

(See Section 9.5 API Structure)

GitHub Repository Link

https://github.com/rajatwarude/DarshanEase.git

Live Demo Link

https://drive.google.com/file/d/1OsG-QbplB0zDcIeaXOM6F_dixQ1VRp75/view?usp=sharing